

IRIS FLOWER CLASSIFICATION USING K-NEAREST NEIGHBOR (KNN)

A Mini Project Report Submitted for Machine Learning Lab

Submitted By

Swati S. Salunkhe

Third Year B.Tech Computer Engineering

Abstract

Machine Learning algorithms are widely used for classification problems in data science and artificial intelligence. This project presents Iris Flower Classification using the K-Nearest Neighbor (KNN) algorithm. The system classifies iris flowers into three species based on their sepal length and sepal width.

The project demonstrates supervised machine learning techniques including data loading, preprocessing, model training, prediction, accuracy evaluation, and graphical visualization using decision boundary graphs. The KNN algorithm predicts the class of new data points based on the nearest neighboring samples.

Introduction

Machine Learning is a branch of Artificial Intelligence that enables systems to learn from data and make predictions. Classification is one of the most important tasks in machine learning.

The Iris dataset is one of the most popular datasets used for learning classification algorithms. It contains measurements of iris flowers belonging to three species:

- Setosa
- Versicolor
- Virginica

In this project, the K-Nearest Neighbor (KNN) algorithm is used to classify iris flowers based on feature similarity.

Objectives

- To understand the KNN classification algorithm.
- To classify iris flower species using machine learning.
- To visualize decision boundaries using graphs.
- To evaluate classification accuracy.

Problem Statement

The objective of this project is to classify iris flowers into different species using the K-Nearest Neighbor algorithm based on sepal measurements.

Dataset Information

The Iris dataset contains:

Feature	Description
Sepal Length	Length of sepal in cm
Sepal Width	Width of sepal in cm
Petal Length	Length of petal in cm
Petal Width	Width of petal in cm
Target	Flower Species

For visualization purposes, only the first two features are used.

Algorithm Used

K-Nearest Neighbor (KNN)

KNN is a supervised machine learning algorithm used for classification and regression.

Working of KNN:

1. Select value of K.
2. Calculate distance between test point and all training points.
3. Select K nearest neighbors.
4. Assign class based on majority voting.

Python Implementation

```
import numpy as np
y = iris.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create KNN Model
k = 5
knn = KNeighborsClassifier(n_neighbors=k)

# Train Model
knn.fit(X_train, y_train)

# Prediction
y_pred = knn.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Classifier Accuracy with k={k}: {accuracy * 100:.2f}%")

# Decision Boundary Visualization
h = 0.02

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.arange(x_min, x_max, h),
    np.arange(y_min, y_max, h)
)

# Predict each point
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot Graph
plt.figure(figsize=(8,6))

plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(
```

```

X[:, 0],
X[:, 1],
c=y,
edgecolor='k'
)

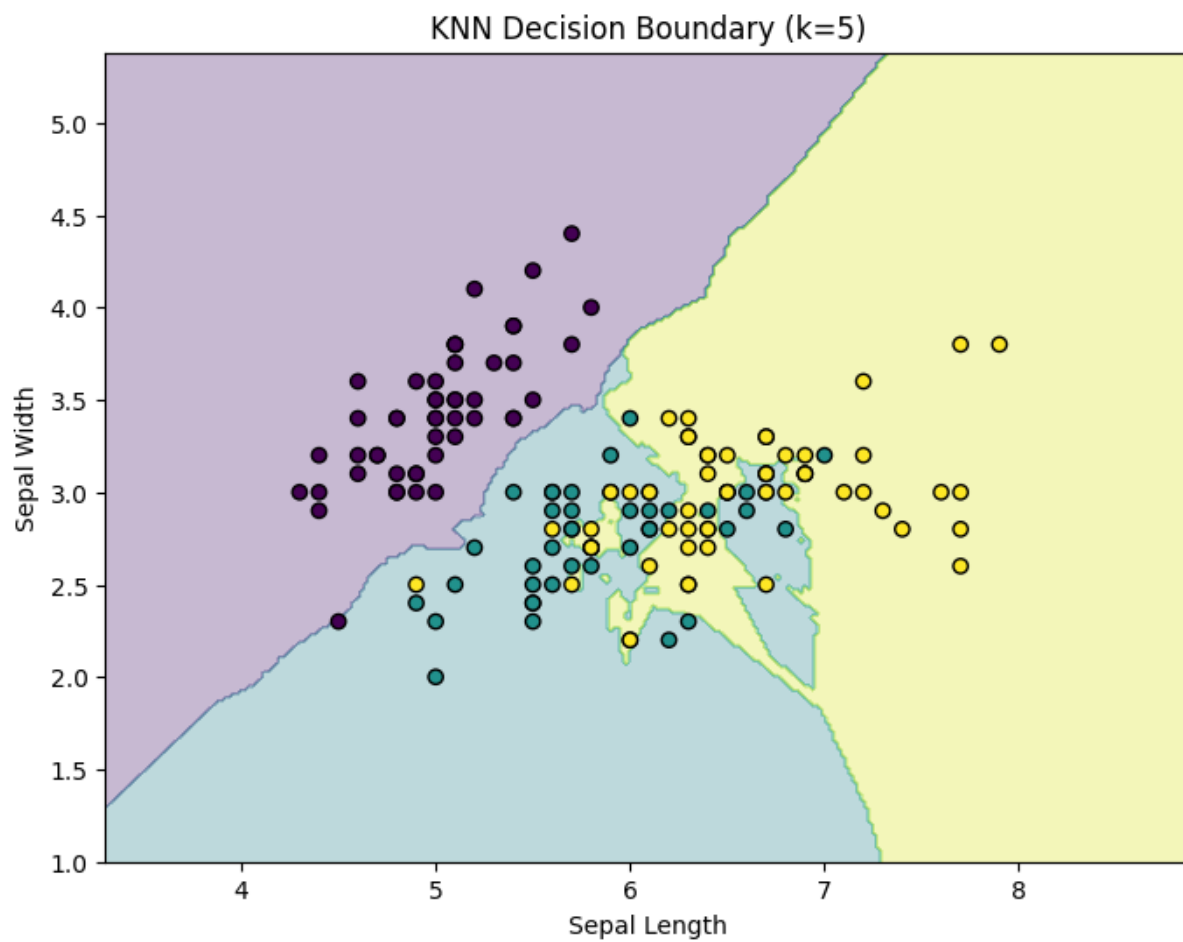
plt.title(f"KNN Decision Boundary (k={k})")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")

plt.show()

```

Output

Classifier Accuracy with k=5: 80.00%



Advantages of KNN

- Simple and easy to implement.
- No training phase required.
- Works well for small datasets.
- Effective for classification problems.

Applications

- Flower classification
- Medical diagnosis
- Recommendation systems
- Pattern recognition
- Image classification

Conclusion

This project successfully implemented the K-Nearest Neighbor algorithm for iris flower classification. The model achieved good classification accuracy and visualized the decision boundaries effectively. The project helps in understanding supervised machine learning and classification concepts.

Future Scope

- Use all four features for better accuracy.
- Apply hyperparameter tuning.
- Compare KNN with SVM and Decision Tree algorithms.
- Use larger datasets for advanced classification.

References

1. Scikit-learn Documentation
2. Python Machine Learning Tutorials
3. Iris Dataset Documentation
4. DBATU Machine Learning Lab Manual